

Runtime Flow

Data engine updates market snapshot. Clock/replay driver dispatches `Runtime.loop_once()`.
`Runtime.loop_once()` reads snapshot and runs bot logic.

Concept Compare

Concept	Mainnet / Testnet / Simnet	Backtest
Who drives whom	<code>Runtime.loop()</code> starts <code>Clock.run()</code> ; wall time wakes Clock; Clock dispatches <code>loop_once()</code> .	<code>Runtime.loop()</code> starts a plain replay loop; the loop pulls data batch, advances <code>ReplayClock</code> , ingests batch, then dispatches <code>loop_once()</code> .
Driver source	Wall time through asyncio event loop.	Prepared replay data through a plain loop.
Wait behavior	<code>asyncio.sleep(...)</code> waits for real time.	No wait; consume next replay batch.
Data arrival	Websocket messages arrive independently of the bot loop.	Replay loop pulls timestamp batches.
Data effect	Updates market buffers only.	Batch timestamp advances replay time; batch events update market buffers.
Time advance	Clock samples wall time after sleep.	Replay clock jumps to batch timestamp.
Same timestamp	Many messages can collapse into latest snapshot before one timer loop.	Many replay events collapse into one timestamp batch.
Loop trigger	Due clock timer dispatches <code>loop_once()</code> .	Batch ingest completes, then due replay timer dispatches <code>loop_once()</code> at most once for that timestamp.
Signaler fire	On new usable bar in snapshot.	On new usable bar in fully ingested batch snapshot.
Snapshot shape	Latest BBO/price plus bars by interval.	Same shape after batch ingest.

Loop count	One per due wall-time runtime timer.	One per due replay-time runtime timer after timestamp batch ingest.
-------------------	--------------------------------------	---

Code Execution Compare

Step	Mainnet / Testnet / Simnet	Backtest
1. Build runtime	Create config, Clock, WsDataEngine, signalers, risk, executor.	Create config, ReplayClock, FileDataEngine, signalers, risk, executor.
2. Data init	Validate config/history access.	Load trusted historical data. Prepare replay events, timestamp-indexed indicator arrays/signals, and timestamp batches. No live-feed readiness gate.
3. Signaler init/seed	Load each signaler's final required seed count. The signaler owns the 10-row protective allowance. Calculate initial indicator state, for example EMA.	Seed from past closed bars only. Precomputed indicators/signals are timestamp-indexed; decision timestamp T can read only values derived from bars closed at or before T.
4. Start, pre-main-loop	Start websocket, subscribe to BBO and required candle intervals, wait for two valid BBO samples with timeout, seed signalers, register runtime timer.	Verify replay batches ready, seed signalers from past closed bars, register runtime timer.
5. Enter main loop	Runtime.loop() calls Clock.run().	Runtime.loop() calls replay loop.
6. Main loop	See steps 6a to 6f.	See steps 6a to 6f.
6a. Pull work	No data batch is pulled. Live waits for next due timer.	time_batch = next(replay_batches). Bar events are timestamped at candle close time, not candle open time.
6b. Wait	await asyncio.sleep(...) until timer is due.	No wait. The plain loop continues if a batch exists.
6c. Advance time	Sample wall time and advance Clock.	Set ReplayClock to time_batch.ts_ms.

6d. Update snapshot	No loop-owned update. Websocket task already updated buffers as messages arrived.	Ingest every event in time_batch into snapshot.
6e. Same-time handling	Latest BBO wins before timer fires; bars stay keyed by interval and timestamp.	All same-timestamp events are ingested before one loop dispatch; snapshot must only expose data available at that replay timestamp.
6f. Trigger bot loop	Clock dispatches due runtime timer.	Replay clock dispatches due runtime timer after batch ingest, at most once for this timestamp batch.
7. loop_once()	Read snapshot. Process exit/risk/signaler/executor.	Same, but snapshot is fully replay-batch updated.

Same Timestamp Example

05:00:00 1m replay event

05:00:00 1h completed bar event

Backtest treats this as:

one timestamp batch

one replay clock advance to 05:00:00

one full snapshot update

one Runtime.loop_once() dispatch at most

The 1m bar in that batch is the candle that closed at 05:00:00. The 1h bar is the candle that closed at 05:00:00. loop_once() sees both only after they are closed and ingested.

Not:

two clock advances

two Runtime.loop_once() calls